



Universidad Nacional Autonoma de Mexico

Facultad de Ingenieria

Sistemas Distribuidos

Laboratorio S05

Mexi Banco con Docker Compose

Reporte de ejecucion, pruebas de API y persistencia

Profesor: Oscar Manuel Ruiz Hurtado

Grupo: 2

Alumno: Oscar Ivan Montiel Juarez

Semestre: 2027-1

Fecha: 24 de septiembre de 2026

Ciudad Universitaria, Ciudad de Mexico



Índice

1. Introduccion	2
2. Objetivos	2
3. Preparacion y ejecucion	2
3.1. Obtencion del proyecto	2
3.2. Compose y estado de salud	2
4. Pruebas de la API	3
4.1. Cuentas y transferencia interna	3
4.2. SPEI e idempotencia	3
4.3. Estado de cuenta y notificaciones	4
4.4. Errores de negocio	4
5. Persistencia del volumen	5
5.1. Bajar el sistema sin borrar el volumen	5
5.2. Bajar el sistema borrando el volumen	5
6. Revision del codigo	6
7. Revision de Docker Compose y Dockerfile	6
8. Conclusiones	6



1. Introduccion

En este laboratorio se ejecuto Mexi Banco v05.1, un monolito docente de Spring Boot con una base de datos PostgreSQL. El proposito fue relacionar lo visto sobre Docker Compose con un sistema que almacena estado, se comunica por una red privada y expone una API HTTP.

La practica se realizo a partir de la etiqueta v05.1 del repositorio oficial. Se levantaron los servicios, se recorrieron las operaciones equivalentes a la coleccion de Postman y se verifico el comportamiento del volumen al bajar el sistema con y sin la opcion `-v`.

2. Objetivos

- Levantar una aplicacion y su base de datos mediante Docker Compose.
- Verificar que ambos servicios alcancen el estado *healthy*.
- Probar cuentas, transferencias internas, SPEI, movimientos, notificaciones y errores de negocio.
- Comprobar la idempotencia de un SPEI mediante Idempotency-Key.
- Distinguir entre `docker compose down` y `docker compose down -v`.
- Identificar las capas y los limites entre modulos en el monolito.

3. Preparacion y ejecucion

3.1. Obtencion del proyecto

Se obtuvo la version solicitada del proyecto con los siguientes comandos:

```
git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git
cd mexi-banco
```

La etiqueta es importante porque fija el codigo y la configuracion utilizados durante el laboratorio. Con ello se evita ejecutar una version posterior que pudiera cambiar los endpoints o el comportamiento observado.

3.2. Compose y estado de salud

El sistema se construyo y ejecuto con:

```
docker compose up --build -d
docker compose ps
```



El resultado final mostro ambos servicios saludables:

NAME	SERVICE	STATUS	PORTS
mexi-banco-app-1	app	Up 15 seconds (healthy)	0.0.0.0:8080->8080/tcp
mexi-banco-db-1	db	Up 21 seconds (healthy)	5432/tcp

El estado *healthy* de db se obtiene con `pg_isready`; el de app consulta `/actuator/health`. La aplicacion depende de que la base no solo exista, sino que este lista para atender conexiones. La consulta de salud devolvio:

```
GET /actuator/health -> HTTP 200
{"status":"UP","groups":["liveness","readiness"]}
```

4. Pruebas de la API

Las peticiones de la coleccion se ejecutaron en el mismo orden mediante un verificador HTTP. Se usaron CLABEs nuevas para evitar conflictos con corridas anteriores. La cuenta origen comenzo con 1000, la cuenta destino con 500 y la transferencia interna fue de 200.

4.1. Cuentas y transferencia interna

La apertura de ambas cuentas devolvio 201 Created. Tras transferir 200, los saldos observados fueron 800 para el origen y 700 para el destino.

```
POST /cuentas (origen, saldoInicial=1000) -> HTTP 201
POST /cuentas (destino, saldoInicial=500) -> HTTP 201
POST /transferencias (origen, destino, 200) -> HTTP 201

GET /cuentas/origen -> saldo: 800.0
GET /cuentas/destino -> saldo: 700.0
```

La transferencia es una transaccion local: en una sola unidad se carga el origen, se abona el destino, se registran los movimientos y se crean las notificaciones. Esto es posible porque todos los modulos usan el mismo proceso y la misma base de datos.

4.2. SPEI e idempotencia

Se envio un SPEI de 50 desde el origen a una CLABE externa, con una cabecera de idempotencia. El primer envio y el reintento devolvieron el mismo registro, identificado con el mismo ID y clave de idempotencia:



```
POST /spei, Idempotency-Key: s05-790286848714 -> HTTP 201
{"id":3,"monto":50,"estado":"ENVIADO"}

POST /spei, misma Idempotency-Key -> HTTP 201
{"id":3,"monto":50.0,"estado":"ENVIADO"}

GET /cuentas/origen -> saldo: 750.0
```

El segundo envio no volvio a descontar 50. La clave unica reserva la solicitud antes de cobrar; si la clave ya existe y el SPEI fue enviado, el servicio devuelve la misma solicitud. Por ello el saldo cambio de 800 a 750 una sola vez.

4.3. Estado de cuenta y notificaciones

El estado de cuenta mostro tres movimientos para el origen: deposito inicial por 1000, transferencia enviada por -200 y SPEI enviado por -50. Las notificaciones asociadas confirmaron los dos cargos y el saldo resultante.

```
GET /movimientos?clabe=origen -> HTTP 200
SPEI_ENVIADO          -50.0    saldo resultante: 750.0
TRANSFERENCIA_ENVIADA -200.0  saldo resultante: 800.0
DEPOSITO              1000.0  saldo resultante: 1000.0

GET /notificaciones?clabe=origen -> HTTP 200
SPEI_ENVIADO por 50, saldo resultante 750.00
TRANSFERENCIA_ENVIADA por 200, saldo resultante 800.00
```

4.4. Errores de negocio

Tambien se verificaron los casos de error incluidos en la coleccion. Los resultados se resumen en la Tabla 1.



Cuadro 1: Respuestas obtenidas en casos no validos.

Caso	Estado	Observacion
CLABE inexistente	404	No se encontro la cuenta solicitada.
CLABE repetida	409	La restriccion de unicidad impidio crearla de nuevo.
Cuerpo sin CLABE	400	Fallo la validacion de la peticion.
Saldo insuficiente	422	La regla de negocio impidio el cargo.
Transferencia a la misma cuenta	422	Origen y destino no pueden ser iguales.
SPEI sin Idempotency-Key	400	Falto la cabecera obligatoria.

Finalmente se consulto de nuevo el saldo del origen. Permanecio en 750, por lo que los intentos invalidos no alteraron el estado de la cuenta.

5. Persistencia del volumen

5.1. Bajar el sistema sin borrar el volumen

Despues de crear las operaciones se ejecuto:

```
docker compose down
docker compose up -d
GET /cuentas/002180790286848714 -> HTTP 200
{"id":3,"saldo":750.00}
```

La cuenta permanecio disponible porque `down` elimina los contenedores y la red, pero no elimina el volumen nombrado `mexibanco_datos`. PostgreSQL vuelve a usar ese volumen al crear el nuevo contenedor.

5.2. Bajar el sistema borrando el volumen

Luego se ejecuto:

```
docker compose down -v
docker compose up -d
GET /cuentas/002180790286848714 -> HTTP 404
```

En este caso la cuenta desaparecio. La opcion `-v` elimina tambien el volumen de PostgreSQL; al subir de nuevo, Compose crea uno vacio y la base se inicializa sin los datos anteriores.



6. Revision del codigo

El paquete `src/main/java/mx/mexibanco` se organiza en los modulos `cuenta`, `movimiento`, `transferencia`, `notificacion` y `spei`. Cada modulo mantiene cuatro responsabilidades:

- **Controlador**: recibe HTTP y delega, por ejemplo `CuentaController`.
- **Servicio**: contiene reglas y limites transaccionales, por ejemplo `CuentaService`.
- **Repositorio**: consulta o modifica las tablas del modulo.
- **Entidad**: representa la fila persistida, por ejemplo `Cuenta`.

El servicio de transferencias no accede directamente al repositorio de cuentas: usa el servicio de cuentas para cargar y abonar. Esta separacion marca una costura futura: si el monolito se divide, esa llamada local se transformaria en comunicacion de red y ya no seria posible mantener una sola transaccion de base de datos de la misma forma.

El modulo SPEI implementa la idempotencia con una restriccion unica sobre la clave de idempotencia. Primero reserva la solicitud y despues ejecuta el cargo. Ante un reintento terminado, devuelve el registro existente sin volver a cobrar.

7. Revision de Docker Compose y Dockerfile

El archivo `docker-compose.yml` declara dos servicios. `db` usa `postgres:17-alpine`, recibe la configuracion mediante variables de entorno y monta el volumen nombrado. `app` se construye desde el Dockerfile y publica el puerto 8080. La red por defecto se nombra explicitamente `mexibanco`; dentro de ella la aplicacion se conecta a la base mediante `DB_HOST=db`, es decir, por el nombre del servicio y no por una IP fija.

El Dockerfile es de dos etapas. La primera usa un JDK de Java 21 para descargar dependencias y empaquetar el JAR. La segunda usa un JRE de Java 21 y copia solo el JAR final. Esto separa las herramientas de compilacion de la imagen de ejecucion, disminuyendo lo que se lleva al contenedor final.

8. Conclusiones

Docker Compose permitio declarar y ejecutar la aplicacion y PostgreSQL como un sistema reproducible. Los healthchecks evitaron que la aplicacion comenzara antes de que la base estuviera preparada, y el DNS interno resolvió `db` sin configurar direcciones IP.

La secuencia de pruebas confirmo que las transferencias actualizan los saldos correctamente, que la idempotencia evita descontar un SPEI dos veces y que los errores de negocio no modifican el saldo. Finalmente, el experimento con `down` y `down -v` hizo visible la diferencia entre recrear contenedores y eliminar el estado persistente.